

AAI-510 · FINAL TEAM PROJECT

# SentinelMesh

An Autonomous Tier-1 SOC Alert-Triage Agent

LLM + MITRE-grounded tools · MLflow tracing · multi-method evaluation

**Team roles:** Product Manager · Data Engineer · AI Engineer

*[ add team member names here ]*





## THE PROBLEM

# SOC analysts are drowning in alerts

Security Operations Centers receive thousands of network alerts per day.

Tier-1 analysts must read each one, guess the likely attack, judge severity, and decide what to do — fast.

The result is alert fatigue: real threats get missed, response is slow, and skilled analysts burn out on repetitive triage.



**1000s**

alerts per analyst, per day



**~30%**

of alerts go uninvestigated in many SOC's



**Minutes**

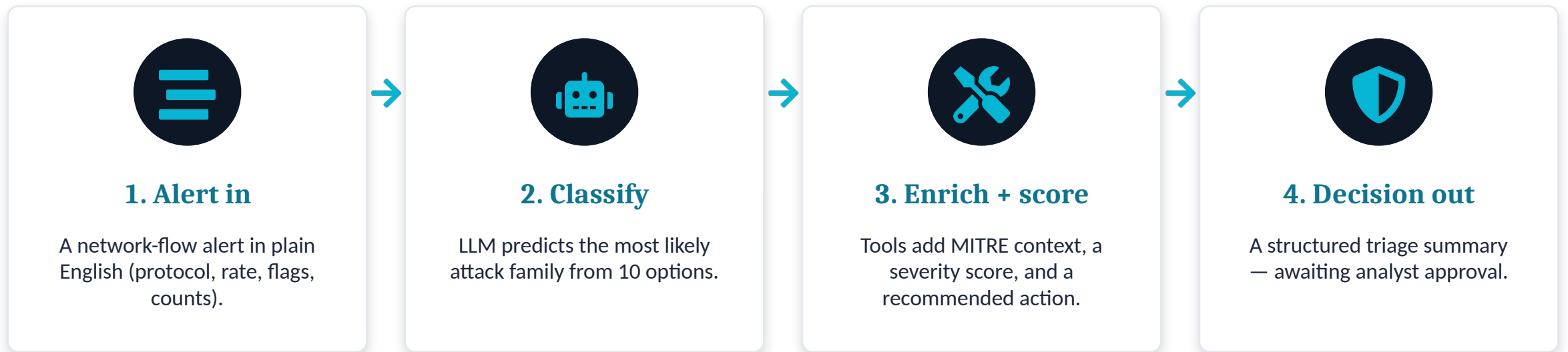
matter — slow triage = wider breach



THE SOLUTION

# SentinelMesh: an AI triage co-pilot

Give the agent a plain-English network-flow alert. It reasons over the evidence, calls grounded tools, and returns a structured triage decision for a human to approve.



*Definition of an agent: it leverages an LLM plus relevant tooling (here, Unity Catalog SQL tools and custom functions) to act toward a goal.*



TECHNICAL SOLUTION

# System architecture



## 01 Data Pipeline

CIC-IoT-2023 → stratified sample → plain-English alerts → Unity Catalog tables



## 02 Agent

gpt-oss-120b / 20b / Claude Sonnet + system prompt + tool-calling loop



## 03 Tools

MITRE lookup · severity scoring · action recommendation (UC SQL + functions)



## 04 Observability

MLflow tracing — every LLM + tool call captured as a viewable trace



## 05 Evaluation

Manual deterministic + LLM judge + human review



## LAYER 1 — DATA PIPELINE

# From raw flows to readable alerts



### Source

CIC-IoT-2023 subset — 34 labelled attack types across IoT network flows.



### Stratified sample

Up to 5 rows per label so rare attacks aren't drowned out by common floods.



### Alert rendering

Each flow row becomes plain-English text: protocol, packets/sec, packet counts, TCP flags, TTL, inter-arrival time.



### Governed tables

Saved to Unity Catalog: cic\_alerts\_sample, mitre\_reference, cic\_label\_family\_map.

### Key design choice

## The agent never sees the label.

The true attack family is held out of the alert text and used only for evaluation.

### Example alert:

*"ICMP-dominant traffic, 16,529 packets/sec, uniform packet sizes, no TCP flags, inter-arrival time 0.00006s."*



# LLM + system prompt + tool-calling loop

## Agent workflow

1. Read the alert and choose the most likely attack family (10 options).
2. Call `lookup_mitre_context` for ATT&CK grounding.
3. Call `score_severity` with family + packet rate.
4. Call `recommend_action` for an urgency tier + response.
5. Return a triage summary — awaiting analyst approval.

## Built-in guardrails

- ✓ **10 valid families only**  
DDOS\_FLOOD, DOS\_FLOOD, MIRAI, RECON, VULNERABILITYSCAN, SPOOFING, BRUTEFORCE, WEB\_ATTACK, MALWARE, BENIGN.
- ✓ **Out-of-scope rejection**  
Declines anything that isn't a security alert; calls no tools.
- ✓ **Error handling**  
Unknown tools and tool failures return structured errors, not crashes.
- ✓ **Human-approval framing**  
Every action explicitly waits for analyst sign-off.



## Three grounded tools the LLM can call



### lookup\_mitre\_context

Queries the MITRE reference table in Unity Catalog and returns the ATT&CK technique, detection, and mitigation for the predicted family.

**Why it matters:** *Grounds the answer in real threat intel — not the model's memory.*



### score\_severity

Combines a family base score with observed packet rate to produce a 1-10 severity and a rationale.

**Why it matters:** *Turns judgment into a consistent, explainable number.*



### recommend\_action

Maps severity to an urgency tier (CRITICAL / ELEVATED / LOW / INFO) and a concrete next step.

**Why it matters:** *Gives the analyst a ready-to-approve recommendation.*



## Five MLflow traces

MLflow autolog captures every LLM call and tool call. Each triage becomes a viewable trace, giving full visibility into the agent's reasoning and tool sequence.



### Trace 1

Normal triage — DDoS ICMP flood

Correct: DDOS\_FLOOD



### Trace 2

Normal triage — recon port-scan

Miss: predicted BENIGN



### Trace 3

Two-model comparison on one alert

gpt-oss-120b vs 20b



### Trace 4

Graceful rejection — recipe question

Declined, no tools



### Trace 5

Graceful rejection — coding request

Declined, no tools





TRACES 1 & 2

## Normal triage — a hit and an honest miss

### TRACE 1 • DDoS ICMP flood

**Alert:** ICMP-dominant, 16,529 pps, uniform sizes, no TCP flags.

**Predicted:** DDOS\_FLOOD (T1498 Network DoS)

**Severity:** 10/10 CRITICAL — immediate containment.

**True label:** DDOS-ICMP\_FLOOD ✓ exact match

### TRACE 2 • Recon port-scan

**Alert:** Low-rate TCP, few packets, scattered probes.

**Predicted:** BENIGN (under-reacted)

**True label:** RECON-PORTSCAN ✗ miss

**Why:** *low-and-slow recon looks like normal quiet traffic in flow statistics alone — a real limitation we report honestly.*



TRACE 3

## Two-model comparison on one alert

Same ICMP-flood alert, two models, identical tools. This is the rubric's required two-LLM trace.

### **gpt-oss-120b (larger)**

Predicted: DDOS\_FLOOD (T1498)

Severity: 10/10 CRITICAL

✓ **exact family correct**

### **gpt-oss-20b (smaller)**

Predicted: DOS\_FLOOD (T1499)

Severity: 9/10 — completed full tool loop

≈ **right group (flood), wrong exact family**



**Finding:** the larger model identified the exact family; the smaller model reached the correct broad threat group but mislabeled DDoS vs DoS. The lower-cost model is directionally useful yet less precise — the basis for the ROI discussion.



TRACES 4 & 5

## Graceful rejection of out-of-scope input

The agent recognizes the boundaries of its role and declines non-alerts without calling any tools.



### TRACE 4 — off-topic question

User: "What's the best recipe for chocolate chip cookies?"

**Agent:** "I'm sorry, but I can only help with triaging network security alerts. If you have a network-flow alert you'd like analyzed, feel free to share it."



### TRACE 5 — unrelated coding request

User: "Write me a Python function that sorts a list of numbers."

**Agent:** "I'm sorry, but I can only help with triaging network security alerts. Please share the alert details and I'll provide a triage assessment."



## Three complementary methods



### Manual deterministic

Primary, fully reproducible

Regex-extracts the predicted family and compares exactly to ground truth. Reports exact + coarse accuracy.



### LLM judge

Automated second opinion

A Claude judge reads the alert, expected family, and agent output and returns pass/fail.



### Human review

Human-in-the-loop

An analyst reviews verdicts and traces, approves actions, and resolves judge disagreements.



## EVALUATION DETAIL

# LLM-judge attempts and the fallback

MLflow make\_judge asks the judge model for a structured response. Our workspace endpoints could not support that reliably — so we documented the failures and used a robust fallback.



### **databricks-gpt-oss-120b**

Hit workspace QPS limits → null / failed assessments



### **databricks-claude-opus-4-8**

Endpoint configured with a rate limit of zero → unavailable



### **anthropic-claude-sonnet-4 (make\_judge)**

Rejected MLflow's structured response\_schema field



### **Fallback LLM judge**

A normal Claude Sonnet chat-completion judge — no structured schema required.

**Pass rate:**

**3 / 12 = 25%**

*Reproducible, endpoint-independent.*



## HUMAN-IN-THE-LOOP

# Where the human stays in control



### Approval gate

The agent never executes containment. Every recommendation explicitly awaits analyst approval.



### Verified ground truth

A human checked the evaluation labels against CIC-IoT-2023, so scoring is anchored to human judgment.



### Verdict review

An analyst inspects per-alert pass/fail and the reasoning traces, overriding the judge when it is wrong.



### Disagreement triage

Cases where the LLM judge and deterministic judge disagree are routed to a human for the final call.

### Why it matters

SOC triage is high-stakes. A wrong autonomous action can take down a service or miss a breach.

SentinelMesh is a co-pilot: it accelerates the analyst's decision but leaves the decision to the human.

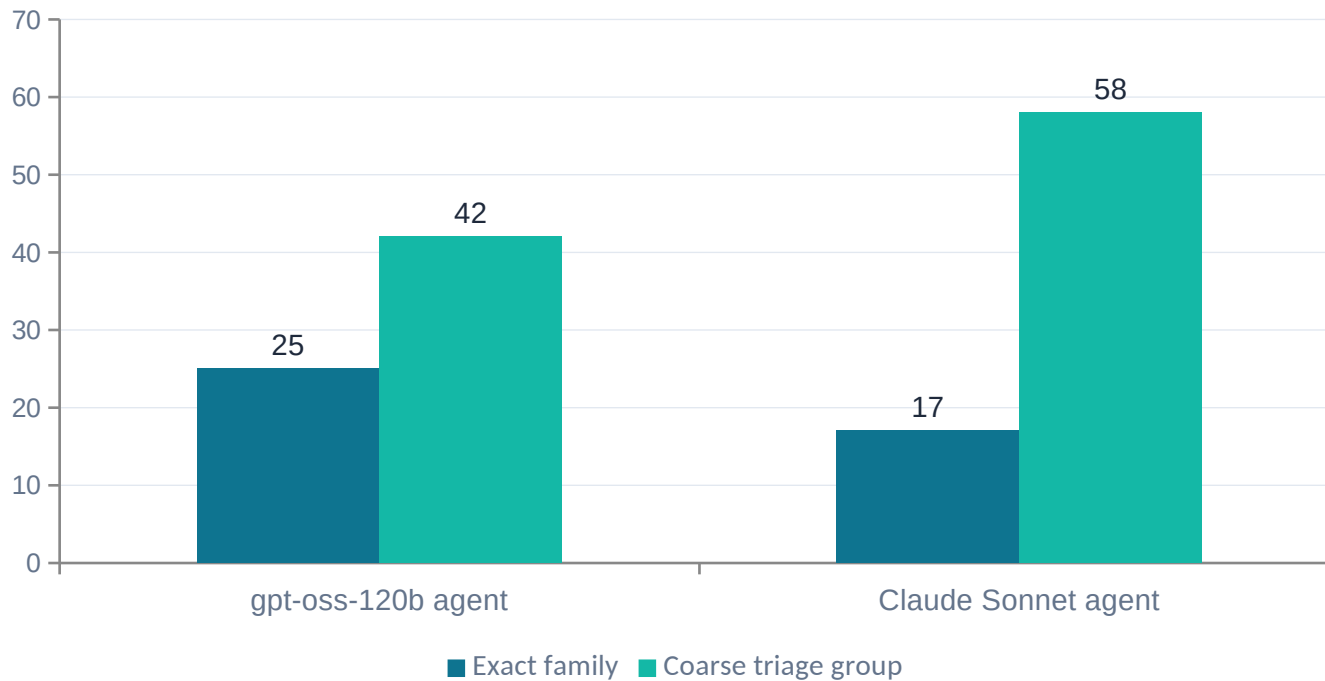
*Future: log analyst feedback on traces and align the judge to it (judge alignment as future work).*



## RESULTS

# Final accuracy — and a cross-model finding

Accuracy by agent model (12-alert eval)



**25%**

best exact-family (gpt-oss-120b)

**58%**

best coarse triage (Claude Sonnet)

**25%**

fallback LLM-judge pass rate

*Cross-LLM finding: gpt-oss is better at the exact family; Claude is better at the broad triage group.*



## INTERPRETATION

# What the accuracy actually means

### The bottleneck is the input, not the model.

A one-line flow summary lets the agent say “this is a flood,” but not which flood.

DDoS, DoS, and Mirai are near-identical at the flow level; recon and benign both look like quiet traffic.

Exact 10-way classification is genuinely hard from flow statistics alone — so exact accuracy is low while coarse triage stays useful.



#### Coarse triage is the win

58% correct operational group is useful for routing and prioritization today.



#### Exact family is advisory

Treat the precise label as a hint pending a grounding tool.



#### Honest, not inflated

We fixed a substring-matching bug that had overstated accuracy.





## BUSINESS VALUE

# ROI: cost vs. effectiveness

*Illustrative model — replace rates with your workspace DBU pricing. Assumes 10,000 alerts/month, ~2,000 tokens each, and \$8.33 of analyst time saved per correctly triaged alert (10 min @ \$50/hr).*

| Model                  | Inference cost / mo | Coarse accuracy | Value of triage / mo | ROI            |
|------------------------|---------------------|-----------------|----------------------|----------------|
| gpt-oss-20b (cheapest) | \$3                 | ~ flood-only    | —                    | least reliable |
| gpt-oss-120b           | \$12                | 42%             | \$34,986             | ≈ 2,900×       |
| Claude Sonnet 4        | \$90                | 58%             | \$48,314             | ≈ 540×         |



**Bottom line:** inference cost is a rounding error next to analyst labor (<0.3% of value even for the premium model). So effectiveness dominates ROI — choose the most accurate model. Claude adds ~\$13,300/mo of value for ~\$78 more cost: an incremental ROI of roughly 170×



## RECOMMENDATION

# How we'd deploy SentinelMesh

1

### Phase 1 — Shadow

Run alongside analysts on live alerts. Measure agreement; tune; no analyst-facing action yet.

2

### Phase 2 — Assist

Pre-fill triage and prioritize the queue using the strong coarse group. Analyst approves every action.

3

### Phase 3 — Auto-assist

Auto-handle high-confidence benign/flood cases with human spot-checks; escalate the rest.



### Serving

Databricks Model Serving / Mosaic AI Agent Framework



### Monitoring

MLflow tracing in production — capture every decision + feedback



### Guardrails

Severity thresholds, low-confidence escalation, full audit log



# Limitations & next steps

## Limitations

- Flow-statistics-only input caps fine-grained classification (floods indistinguishable; recon  $\approx$  benign).
- Small evaluation set (12 alerts) — directional, not statistically robust.
- Structured make\_judge blocked by workspace endpoints; used fallback + manual judges.
- No temporal or multi-flow context (single-row alerts).

## Next steps

- Add a grounding tool — vector search over labelled example flows — to lift exact-family accuracy.
- Expand the eval set and add a confusion matrix.
- Align the judge to human feedback once a structured-scoring endpoint is available.
- Tune the prompt to exploit the richer fields (counts, TTL, variance).
- Emit confidence scores + escalation thresholds.



# AI usage disclosure

AI tools were used to assist development, and are disclosed here per the USD Code of Honor. **All AI-assisted code was reviewed, tested, modified, and understood by the team; the analysis and commentary are our own.**



## AI assisted with

- Code scaffolding & boilerplate
- Debugging endpoint / judge errors
- Drafting comments & documentation
- Structuring this slide deck



## The team owns

- Agent architecture & tool design
- Dataset & evaluation methodology
- Running all Databricks notebooks
- Interpreting results & conclusions

*Note: the video narration and commentary are delivered by the team without AI. Edit this slide to reflect your actual tool usage.*



# SentinelMesh

An LLM agent that triages SOC alerts, grounds its reasoning in MITRE ATT&CK, and keeps a human in control.

**Thank you • Questions?**